CLUSTER COMPUTING

PROJECT REPORT

TITLE

Telco Customer Churn Prediction and Analytics Using Apache Spark, PySpark, Scala, and Spark MLlib

Submitted by

Junaid Raza

Registration No 12315227

Programme and Section B.Tech(CSE) and K3P23AZ

Course Code INT 315

Under the Guidance of

(Priyanka Kumari (UID-32353))

Discipline of CSE/IT

Lovely School of Computer Science and Engineering

Lovely Professional University, Phagwara





CERTIFICATE

This is to certify that Junaid Raza bearing Registration no. 12315227 has complete INT-315 project titled, **"Telco Customer Churn Prediction and Analytics Using Apache Spark, PySpark, Scala and Spark MLlib"** under my guidance and supervision. To the best of my knowledge, the present work is the result of his/her original development, effort and study.

School of Computer Science & Engineering
Lovely Professional University
Phagwara, Punjab.

DECLARATION

I, Junaid Raza, Student of Computer Science & Engineering under CSE/IT Discipline at, Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: 24/10/2026 Signature

Registration No.*12315227* *Junaid Raza*

Acknowledgement

I would like to express my sincere gratitude to my faculty guide for providing continuous support, guidance, and motivation throughout the completion of this project. I am also thankful to the Lovely Professional University for providing the necessary resources and learning environment. Finally, I thank my family and friends for their encouragement and support.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objective of the Project
4. Dataset Description
5. Dataset Source
6. Tools & Technologies Used
7. Implementation / Methodology
8. Complete Code
9. Execution Screenshots
10. Output / Result Screenshots
11. Result Analysis
12. Conclusion

1. Introduction

Customer churn – the loss of subscribers to a competitor or to cancellation – is one of the most expensive problems in the telecommunications industry, because acquiring a new customer costs several times more than retaining an existing one. Telecom operators therefore invest in analytics that can find customers who are likely to leave early enough for a retention team to act.

This project builds an end-to-end big data analytics pipeline for the IBM Telco Customer Churn dataset using **Apache Spark** as the cluster computing framework. Data ingestion, cleaning, feature engineering, SQL analytics, machine learning and scoring are implemented in **PySpark** with **Spark SQL** and **Spark MLlib**, while a **Scala** companion module demonstrates typed Datasets, immutable collections, higher-order functions and lazy evaluation. All results are persisted as columnar **Parquet** and JSON files, and a lightweight Node.js/Express API serves them to a dashboard without any database server.

The project re-engineers an earlier prototype built on pandas, scikit-learn and MongoDB: the document database is removed completely and replaced by Spark processing with Parquet/JSON storage. The pipeline is designed for distributed execution but is run here in Spark's standalone local mode (`local[*]`), where one JVM uses all CPU cores as parallel task slots. Moving to a multi-node cluster would mainly require changing the master URL and the input/output paths (for example to HDFS or cloud storage).

2. Problem Statement

A telecom provider holds records for thousands of subscribers, but it cannot tell in advance which customers will cancel, which service or contract factors drive cancellations, or how much recurring revenue is at stake. Single-machine tools such as pandas and scikit-learn are adequate for a small sample, yet they keep the whole dataset in one process's memory, tie the analytics to an operational database, and do not scale out as call-detail and billing data grow.

The problem addressed in this project is therefore to **design and implement a distributed, database-free analytics pipeline** that (a) predicts which customers are likely to churn, (b) explains the main churn drivers through SQL analytics, (c) groups customers into behavioural segments, and (d) delivers the results to a dashboard through a decoupled serving layer – while evaluating the models honestly on unresampled, held-out data.

3. Objective of the Project

The main objectives of this project are:

- To ingest and validate the Telco churn dataset in Apache Spark using an explicit schema, and to clean and engineer features at DataFrame scale.
- To perform exploratory analytics with Spark SQL and DataFrame aggregations, inspect the Catalyst query plan, and measure the benefit of in-memory caching.
- To build a leak-free Spark ML Pipeline (StringIndexer, OneHotEncoder, VectorAssembler, StandardScaler) and compare Logistic Regression, Decision Tree, Random Forest, Gradient-Boosted Trees and Linear SVM with 3-fold cross-validation.
- To evaluate models with ROC-AUC, PR-AUC, accuracy, precision, recall, F1 and confusion matrices on a stratified hold-out set.
- To score every customer, assign High / Medium / Low churn-risk levels and estimate monthly revenue at risk.
- To segment customers with K-Means clustering and to estimate MonthlyCharges with Linear Regression (R^2, RMSE, MAE).
- To demonstrate Scala features (type inference, case classes, typed Datasets, immutable collections, pattern matching, lazy streams) alongside Spark.
- To persist all outputs as Parquet/JSON marts and serve them through an in-memory Express API with no database dependency.

4. Dataset Description

The project uses the **IBM Telco Customer Churn** dataset: a customer-level table for a fictional telecommunications company that provides home phone and Internet services. It contains **7,043 customer records** and **21 columns** (20 predictors and one target). The target column `Churn` is "Yes" for the **1,869 customers (26.54%)** who left within the last month and "No" for the remaining 5,174 customers, so the classes are imbalanced (about 3 : 1).

Eleven records have a blank `TotalCharges` value; all of them are brand-new customers with `tenure = 0`, so the value is imputed as `MonthlyCharges × tenure = 0`. Numeric ranges are tenure 0–72 months, MonthlyCharges 18.25–118.75 and TotalCharges 18.80–8,684.80. The `customerID` column is retained for joining and reporting but excluded from model training.

Table 4.1 – Attributes of the raw dataset

Group	Columns	Type / values
Identifier	customerID	String, unique (e.g. 7590-VHVEG)
Demographics	gender, SeniorCitizen, Partner, Dependents	Male/Female; 0/1; Yes/No
Account	tenure, Contract, PaperlessBilling, PaymentMethod	Months 0–72; Month-to-month / One year / Two year; Yes/No; 4 payment methods
Phone services	PhoneService, MultipleLines	Yes / No / No phone service
Internet services	InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies	DSL / Fiber optic / No; add-ons Yes / No / No internet service
Charges	MonthlyCharges, TotalCharges	Double (currency units)
Target	Churn	Yes / No

Table 4.2 – Features engineered in Spark (row-level, no leakage)

Feature	Definition
AvgMonthlySpend	TotalCharges / (tenure + 1)
ChargeDifference	MonthlyCharges − AvgMonthlySpend (rate escalation)
TotalServices	Count of the six add-on services (security, backup, protection, support, streaming TV/movies) equal to "Yes"
StreamingRatio	(StreamingTV = Yes + StreamingMovies = Yes) / (TotalServices + 1)
TenureRatio	tenure / 72
TenureGroup	Bins: 0–12, 13–24, 25–48, 49–72 months

After cleaning and feature engineering the curated table has 27 columns. The classifiers use 8 numeric and 17 categorical inputs, which the Pipeline expands to a **72-dimensional** scaled feature vector.

5. Dataset Source

The dataset is the publicly available **IBM Telco Customer Churn** sample dataset, originally published by IBM (Cognos Analytics sample data) and widely distributed through Kaggle (uploaded by "blastchar"). It is anonymised: it contains customer IDs but no names, addresses or payment details.

Original source and download links:

> Kaggle: https://www.kaggle.com/datasets/blastchar/telco-customer-churn
> https://github.com/junaidraza134/Telco-Customer-Churn-Prediction

File used in this project: `WA_Fn-UseC_-Telco-Customer-Churn.csv`, stored in `data/raw/` (7,043 data rows, 21 columns). The row count, column count and uniqueness of `customerID` are validated by the ingestion script at run time.

6. Tools & Technologies Used

Category	Technology	Version / role
Cluster computing framework	Apache Spark	3.5.3, standalone / local mode `local[*]`; driver memory 4 GB; Adaptive Query Execution on
Spark components	Spark SQL (Catalyst, Tungsten), Spark ML (MLlib DataFrame API), Structured DataFrames	Queries, feature transformers, estimators, evaluators, CrossValidator
Programming languages	Python (PySpark)	Python 3.12.3 with PySpark 3.5.3 – main pipeline
	Scala	2.12.18 – typed Dataset and functional companion module (build.sbt provided)
	JavaScript	Node.js 22 – API server
Runtime	OpenJDK	21.0.10 (Spark 3.5 also supports Java 8/11/17)
Storage formats	Apache Parquet (Snappy), CSV, JSON	Columnar analytical mart, portable exports, API artefacts
Python libraries	pandas, matplotlib, seaborn	Export to JSON, charts, correlation heatmap
ML algorithms	Logistic Regression, Decision Tree, Random Forest, GBT, LinearSVC, K-Means, Linear Regression, PCA	pyspark.ml.classification / clustering / regression / feature
Serving layer	Express 5.3, cors 2.8	In-memory file-based REST API (no database)
Presentation	React 18 + Vite + Tailwind CSS	Existing dashboard consuming the API
Execution environment	Ubuntu Linux (x86-64)	Runs on Windows/macOS with Java installed

7. Implementation / Methodology

The implementation follows a four-tier architecture: raw data, distributed processing, a persistent analytical data mart, and a lightweight serving layer. Tiers communicate only through files (Parquet and JSON), which keeps the stages independent and re-runnable.

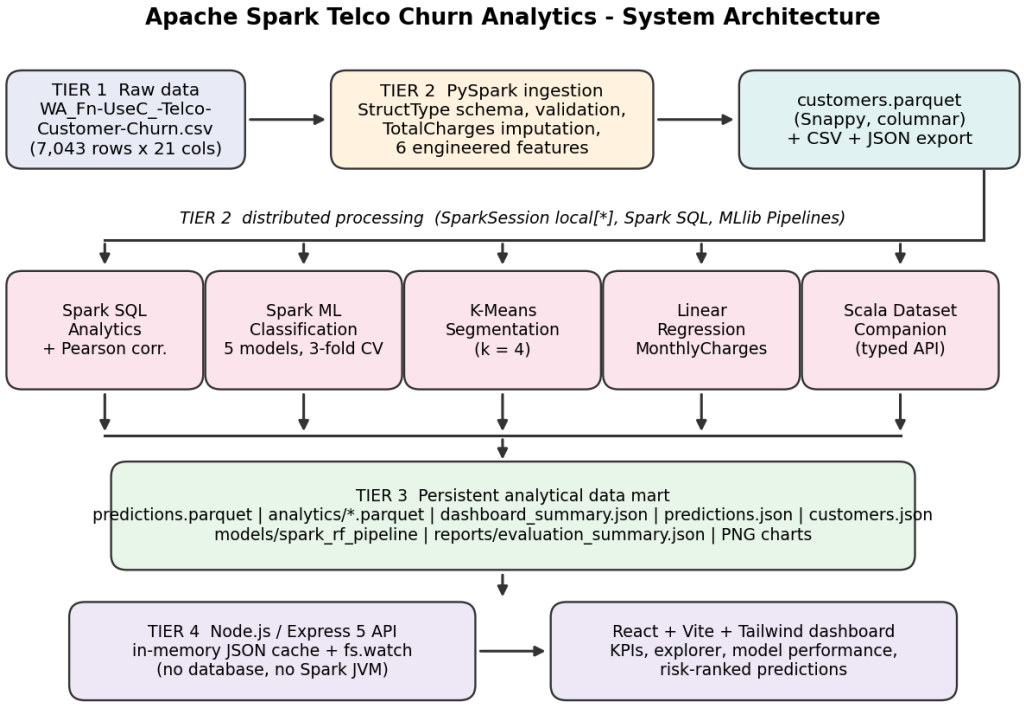


Figure 7.1 – System architecture of the Spark-based churn analytics platform

Step 1 – Spark session and environment. A reusable `get_spark()` factory builds a SparkSession with `master("local[*]")`, 4 GB driver memory, 8 shuffle partitions (appropriate for a small dataset) and Adaptive Query Execution. The driver converts code into a DAG of stages, and tasks run on worker threads inside the same JVM.

Step 2 – Ingestion and validation (spark_ingest.py). The CSV is read with an explicit `StructType` schema instead of schema inference. The script asserts 7,043 rows, 21 columns and no duplicate `customerID`, converts blank `TotalCharges` to null, imputes them as `MonthlyCharges × tenure`, and casts the column to double.

Step 3 – Feature engineering. Six features (Table 4.2) are computed from the same row only, so no global statistic can leak from the test set into training. The curated table is written to `customers.parquet` (Snappy), a CSV copy, and `customers.json` for the API.

Step 4 – Spark SQL analytics (spark_analytics.py). The Parquet mart is registered as a temporary view and queried with Spark SQL for overall KPIs and churn rate by Contract, InternetService, TenureGroup and PaymentMethod. `explain(True)` prints the parsed,

analysed, optimised and physical plans (column pruning and two-phase hash aggregation). A caching benchmark compares repeated aggregations on an uncached and a `cache()`d DataFrame, and `Correlation.corr` computes a Pearson matrix of five numeric columns, rendered as a heatmap.

Step 5 – Classification pipeline (spark_train.py). The target is encoded as 1.0 (churn) / 0.0 (stay). A stratified 80/20 split is made with `sampleBy` (80% of each class, seed 42); the remaining rows form the test set and a join confirms zero customer overlap. One preprocessing chain – `StringIndexer → OneHotEncoder → VectorAssembler → StandardScaler` – is placed inside each `Pipeline` together with the classifier, so encoders and scaler are fitted only on training data (and re-fitted inside every cross-validation fold).

Five estimators are trained: Logistic Regression and Random Forest are tuned with `CrossValidator` (3 folds, ROC-AUC) over `ParamGridBuilder` grids (regParam × elasticNetParam, and maxDepth × numTrees); Decision Tree (depth 5), GBT (50 iterations, depth 5) and LinearSVC (regParam 0.1) use fixed settings. Evaluation uses `BinaryClassificationEvaluator` (ROC-AUC, PR-AUC), `MulticlassClassificationEvaluator` (accuracy, weighted precision/recall/F1) and a confusion matrix computed with RDD operations on (prediction, label) pairs.

Step 6 – Scoring and risk stratification. The tuned Random Forest PipelineModel is saved to `models/spark_rf_pipeline`. It scores all 7,043 customers; the churn probability P is the second element of the probability vector. Risk level = High if $P \geq 0.60$, Medium if $0.30 \leq P < 0.60$, otherwise Low, and Revenue at Risk = P × MonthlyCharges. A `split` column marks whether each customer was in the training or test partition, so optimistic scores on training rows are identifiable. Feature importances are mapped back to readable names through the assembler's ML attribute metadata.

Step 7 – K-Means segmentation (spark_clustering.py). Tenure, MonthlyCharges, TotalCharges, TotalServices and an ordinal contract code are standardised (mean 0, variance 1). K-Means (k = 4, seed 42, `k-means||` initialisation) is compared with k = 2–7 using the Silhouette score and WSSSE; clusters are profiled and projected to two dimensions with PCA.

Step 8 – Linear Regression (spark_regression.py). MonthlyCharges is estimated from the subscribed services (InternetService one-hot and Yes/No flags) on an 80/20 random split. An elastic-net model (regParam 0.3, elasticNetParam 0.8) is compared with an unregularised OLS model fitted by the normal equation, which exposes p-values. Redundant "No phone/internet service" levels are replaced by Yes-flags to avoid a singular design matrix.

Step 9 – Scala companion module. `ChurnApp.scala` reads the Parquet mart into a typed `Dataset[Customer]` (case class encoder), runs typed `groupByKey/mapGroups` aggregations, then hands a collected immutable `List` to `CollectionsDemo` (partition, map/filter, foldLeft, groupBy, closures, pattern matching on tuples, composable rule functions, mutable vs immutable collections). `StreamSimulation` shows lazy evaluation with an infinite `Stream` of call events.

Step 10 – Serving layer (dashboard-api/server.js). Express loads `dashboard_summary.json`, `customers.json`, `predictions.json` and `evaluation_summary.json` into memory on start-up and reloads them automatically with `fs.watch` when Spark rewrites a file. Endpoints `/api/health`, `/api/dashboard`, `/api/customers`, `/api/predictions`, `/api/model` and `/api/feature-importance.png` perform filtering, regex search, sorting and pagination in memory, so web requests never start a Spark job.

Execution order

```
pip install -r spark-pipeline/requirements.txt
python spark-pipeline/spark_ingest.py        # 1. Parquet / CSV / JSON customer mart
python spark-pipeline/spark_analytics.py     # 2. Spark SQL analytics, correlation, summary JSON
python spark-pipeline/spark_train.py         # 3. Spark ML models, evaluation, scoring
python spark-pipeline/spark_clustering.py    # 4a. K-Means segmentation
python spark-pipeline/spark_regression.py    # 4b. Linear Regression
cd scala-companion && sbt run                # 5. Scala companion (or spark-submit churn.jar)
cd dashboard-api && npm install && npm start # 6. API on http://localhost:4000
```

8. Complete Code

The complete source code of the project is listed below, file by file. Python files are in `spark-pipeline/`, Scala files in `scala-companion/`, and the API in `dashboard-api/`.

8.1 requirements.txt

```
pyspark==3.5.3
pandas
matplotlib
seaborn
```

8.2 spark-pipeline/config.py

```
"""Central configuration: paths, constants and shared feature-engineering logic."""
import json
from pathlib import Path

from pyspark.sql import DataFrame, functions as F

ROOT = Path(__file__).resolve().parent.parent
RAW_CSV = ROOT / "data" / "raw" / "WA_Fn-UseC_-Telco-Customer-Churn.csv"
PROCESSED = ROOT / "data" / "processed"
ANALYTICS = PROCESSED / "analytics"
API_EXPORTS = PROCESSED / "api_exports"
MODELS = ROOT / "models"
REPORTS = ROOT / "reports"

SEED = 42
TRAIN_FRACTION = 0.8
EXPECTED_ROWS = 7043
HIGH_THRESHOLD = 0.60
MEDIUM_THRESHOLD = 0.30

ADDON_COLS = ["OnlineSecurity", "OnlineBackup", "DeviceProtection",
              "TechSupport", "StreamingTV", "StreamingMovies"]
CATEGORICAL_COLS = ["gender", "SeniorCitizen", "Partner", "Dependents", "PhoneService",
                    "MultipleLines", "InternetService", "OnlineSecurity", "OnlineBackup",
                    "DeviceProtection", "TechSupport", "StreamingTV", "StreamingMovies",
                    "Contract", "PaperlessBilling", "PaymentMethod", "TenureGroup"]
NUMERIC_COLS = ["tenure", "MonthlyCharges", "TotalCharges", "AvgMonthlySpend",
                "ChargeDifference", "TotalServices", "StreamingRatio", "TenureRatio"]


def ensure_dirs():
    for d in (PROCESSED, ANALYTICS, API_EXPORTS, MODELS, REPORTS):
        d.mkdir(parents=True, exist_ok=True)


def write_json(path, obj):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w", encoding="utf-8") as fh:
        json.dump(obj, fh, indent=2, default=str)


def add_engineered_features(df: DataFrame) -> DataFrame:
    """Row-level features only (no global statistics), so no train/test leakage."""
    total_services = sum(F.when(F.col(c) == "Yes", 1).otherwise(0) for c in ADDON_COLS)
    streaming = (F.when(F.col("StreamingTV") == "Yes", 1).otherwise(0)
                 + F.when(F.col("StreamingMovies") == "Yes", 1).otherwise(0))
    return (df
            .withColumn("AvgMonthlySpend", F.col("TotalCharges") / (F.col("tenure") + 1))
            .withColumn("ChargeDifference", F.col("MonthlyCharges") - F.col("AvgMonthlySpend"))
            .withColumn("TotalServices", total_services.cast("int"))
            .withColumn("StreamingRatio", streaming / (F.col("TotalServices") + 1))
            .withColumn("TenureRatio", F.col("tenure") / F.lit(72.0))
            .withColumn("TenureGroup",
                        F.when(F.col("tenure") <= 12, "0-12")
                         .when(F.col("tenure") <= 24, "13-24")
                         .when(F.col("tenure") <= 48, "25-48")
                         .otherwise("49-72")))
```

8.3 spark-pipeline/spark_session.py

```
"""Reusable SparkSession factory (standalone / local mode)."""
from pyspark.sql import SparkSession


def get_spark(app_name: str = "TelcoChurnSpark") -> SparkSession:
    spark = (SparkSession.builder
             .master("local[*]")
             .appName(app_name)
             .config("spark.driver.memory", "4g")
```

```
            .config("spark.sql.shuffle.partitions", "8")
            .config("spark.sql.adaptive.enabled", "true")
            .config("spark.ui.showConsoleProgress", "false")
            .getOrCreate())
    spark.sparkContext.setLogLevel("ERROR")
    return spark
```

8.4 spark-pipeline/spark_ingest.py

```
"""Stage 1 - Ingestion: schema enforcement, cleaning, feature engineering, Parquet/JSON mart."""
from pyspark.sql import functions as F
from pyspark.sql.types import (DoubleType, IntegerType, StringType, StructField, StructType)

import config
from spark_session import get_spark

SCHEMA = StructType([
    StructField("customerID", StringType(), False),
    StructField("gender", StringType(), True),
    StructField("SeniorCitizen", IntegerType(), True),
    StructField("Partner", StringType(), True),
    StructField("Dependents", StringType(), True),
    StructField("tenure", IntegerType(), True),
    StructField("PhoneService", StringType(), True),
    StructField("MultipleLines", StringType(), True),
    StructField("InternetService", StringType(), True),
    StructField("OnlineSecurity", StringType(), True),
    StructField("OnlineBackup", StringType(), True),
    StructField("DeviceProtection", StringType(), True),
    StructField("TechSupport", StringType(), True),
    StructField("StreamingTV", StringType(), True),
    StructField("StreamingMovies", StringType(), True),
    StructField("Contract", StringType(), True),
    StructField("PaperlessBilling", StringType(), True),
    StructField("PaymentMethod", StringType(), True),
    StructField("MonthlyCharges", DoubleType(), True),
    StructField("TotalCharges", StringType(), True),   # blanks present -> cast after cleaning
    StructField("Churn", StringType(), True),
])


def main():
    config.ensure_dirs()
    spark = get_spark("TelcoChurnIngestion")

    raw = spark.read.csv(str(config.RAW_CSV), header=True, schema=SCHEMA)
    n_rows, n_cols = raw.count(), len(raw.columns)
    n_dupes = n_rows - raw.dropDuplicates(["customerID"]).count()
    print(f"[INGEST] rows={n_rows}  columns={n_cols}  duplicate customerIDs={n_dupes}")
    assert n_rows == config.EXPECTED_ROWS and n_cols == 21 and n_dupes == 0, "Validation failed"

    # Data-quality audit: blank TotalCharges become null, then are imputed.
    cleaned = raw.withColumn("TotalCharges", F.when(F.trim("TotalCharges") == "", None)
                             .otherwise(F.trim("TotalCharges")).cast("double"))
    n_null = cleaned.filter(F.col("TotalCharges").isNull()).count()
    print(f"[INGEST] blank/null TotalCharges before imputation: {n_null}")
    cleaned = cleaned.withColumn("TotalCharges",
                                 F.coalesce("TotalCharges", F.col("MonthlyCharges") * F.col("tenure")))
    assert cleaned.filter(F.col("TotalCharges").isNull()).count() == 0

    df = config.add_engineered_features(cleaned)
    assert df.count() == config.EXPECTED_ROWS
    df.printSchema()

    df.write.mode("overwrite").parquet(str(config.PROCESSED / "customers.parquet"))
    df.coalesce(1).write.mode("overwrite").option("header", True).csv(str(config.PROCESSED / "customers_csv"))
    df.toPandas().to_json(config.API_EXPORTS / "customers.json", orient="records", indent=1)
    print(f"[INGEST] wrote Parquet, CSV and JSON marts ({df.count()} records)")
    spark.stop()


if __name__ == "__main__":
    main()
```

8.5 spark-pipeline/spark_analytics.py

```
"""Stage 2 - Spark SQL analytics, caching benchmark, Catalyst plan and Pearson correlation."""
import time

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.stat import Correlation

import config
```

```
from spark_session import get_spark

CORR_COLS = ["tenure", "MonthlyCharges", "TotalCharges", "AvgMonthlySpend", "ChargeDifference"]


def churn_by(spark, column):
    return spark.sql(f"""
        SELECT {column} AS segment, COUNT(*) AS customers,
               SUM(CASE WHEN Churn = 'Yes' THEN 1 ELSE 0 END) AS churned,
               CAST(ROUND(100 * AVG(CASE WHEN Churn = 'Yes' THEN 1.0 ELSE 0.0 END), 2) AS DOUBLE) AS
churn_rate_pct,
               ROUND(AVG(tenure), 2) AS avg_tenure, ROUND(AVG(MonthlyCharges), 2) AS avg_monthly
        FROM customers GROUP BY {column} ORDER BY churn_rate_pct DESC""")


def main():
    config.ensure_dirs()
    spark = get_spark("TelcoChurnAnalytics")
    df = spark.read.parquet(str(config.PROCESSED / "customers.parquet"))
    df.createOrReplaceTempView("customers")

    kpi = spark.sql("""
        SELECT COUNT(*) AS total_customers,
               SUM(CASE WHEN Churn='Yes' THEN 1 ELSE 0 END) AS churned,
               CAST(ROUND(100*AVG(CASE WHEN Churn='Yes' THEN 1.0 ELSE 0.0 END), 2) AS DOUBLE) AS churn_rate_pct,
               ROUND(AVG(tenure), 2) AS avg_tenure, ROUND(AVG(MonthlyCharges), 2) AS avg_monthly,
               ROUND(AVG(TotalCharges), 2) AS avg_total FROM customers""").first().asDict()
    print("[ANALYTICS] KPIs:", kpi)

    results = {}
    for col in ["Contract", "InternetService", "TenureGroup", "PaymentMethod"]:
        sdf = churn_by(spark, col)
        print(f"\n[ANALYTICS] Churn rate by {col}")
        sdf.show(truncate=False)
        results[col] = [r.asDict() for r in sdf.collect()]
        if col == "Contract":
            sdf.write.mode("overwrite").parquet(str(config.ANALYTICS / "contract_churn.parquet"))

    print("[ANALYTICS] Catalyst plans for the Contract aggregation (df.explain):")
    churn_by(spark, "Contract").explain(True)

    # In-memory benchmark: repeated aggregation, uncached vs cached DataFrame.
    def timed(frame, repeats=5):
        t0 = time.time()
        for _ in range(repeats):
            frame.groupBy("Contract", "InternetService").avg("MonthlyCharges").collect()
        return time.time() - t0
    t_uncached = timed(df)
    cached = df.cache(); cached.count()
    t_cached = timed(cached)
    print(f"[ANALYTICS] 5 repeated aggregations - uncached: {t_uncached:.3f}s, cached: {t_cached:.3f}s")

    # Pearson correlation (numeric columns only; correlation != causation).
    vec = VectorAssembler(inputCols=CORR_COLS, outputCol="features").transform(df).select("features")
    corr = Correlation.corr(vec, "features", "pearson").head()[0].toArray()
    print("\n[ANALYTICS] Pearson correlation matrix:")
    print(" " * 18 + "".join(f"{c[:15]:>16}" for c in CORR_COLS))
    for name, row in zip(CORR_COLS, corr):
        print(f"{name:<18}" + "".join(f"{v:>16.3f}" for v in row))
    config.write_json(config.REPORTS / "correlation_matrix.json",
                      {"columns": CORR_COLS, "matrix": corr.round(4).tolist()})
    plt.figure(figsize=(7, 5.5))
    sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1,
                xticklabels=CORR_COLS, yticklabels=CORR_COLS)
    plt.title("Pearson Correlation of Numeric Features (Spark ML)")
    plt.tight_layout(); plt.savefig(config.REPORTS / "correlation_heatmap.png", dpi=150); plt.close()

    summary = {"kpis": kpi, "contractChurn": results["Contract"],
               "internetChurn": results["InternetService"],
               "tenureGroupChurn": results["TenureGroup"],
               "paymentChurn": results["PaymentMethod"],
               "cachingBenchmarkSeconds": {"uncached": round(t_uncached, 3), "cached": round(t_cached, 3)}}
    config.write_json(config.ANALYTICS / "dashboard_summary.json", summary)
    spark.stop()


if __name__ == "__main__":
    main()
```

8.6 spark-pipeline/spark_train.py

```
"""Stage 3 - Spark ML classification: leak-free Pipeline, CV tuning, evaluation and scoring."""
import json
import time

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
import pandas as pd
from pyspark.ml import Pipeline
from pyspark.ml.classification import (DecisionTreeClassifier, GBTClassifier, LinearSVC,
                                       LogisticRegression, RandomForestClassifier)
from pyspark.ml.evaluation import BinaryClassificationEvaluator, MulticlassClassificationEvaluator
from pyspark.ml.feature import OneHotEncoder, StandardScaler, StringIndexer, VectorAssembler
from pyspark.ml.functions import vector_to_array
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder
from pyspark.sql import functions as F

import config
from spark_session import get_spark


def build_prep_stages():
    """StringIndexer -> OneHotEncoder -> VectorAssembler -> StandardScaler (no target involved)."""
    idx_cols = [f"{c}_idx" for c in config.CATEGORICAL_COLS]
    ohe_cols = [f"{c}_ohe" for c in config.CATEGORICAL_COLS]
    indexers = StringIndexer(inputCols=config.CATEGORICAL_COLS, outputCols=idx_cols, handleInvalid="keep")
    encoder = OneHotEncoder(inputCols=idx_cols, outputCols=ohe_cols, handleInvalid="keep")
    assembler = VectorAssembler(inputCols=config.NUMERIC_COLS + ohe_cols, outputCol="raw_features")
    scaler = StandardScaler(inputCol="raw_features", outputCol="features", withMean=True, withStd=True)
    return [indexers, encoder, assembler, scaler]


def confusion_matrix(pred_df):
    """Confusion matrix computed with RDD operations on (prediction, label) pairs."""
    counts = pred_df.select("prediction", "label").rdd.map(lambda r: (int(r[0]), int(r[1]))).countByValue()
    tn, fp = counts.get((0, 0), 0), counts.get((1, 0), 0)
    fn, tp = counts.get((0, 1), 0), counts.get((1, 1), 0)
    return {"tn": tn, "fp": fp, "fn": fn, "tp": tp}


def evaluate(name, pred_df, best_params, fit_seconds):
    roc = BinaryClassificationEvaluator(metricName="areaUnderROC").evaluate(pred_df)
    pr = BinaryClassificationEvaluator(metricName="areaUnderPR").evaluate(pred_df)
    mc = MulticlassClassificationEvaluator()
    acc = mc.setMetricName("accuracy").evaluate(pred_df)
    wp = mc.setMetricName("weightedPrecision").evaluate(pred_df)
    wr = mc.setMetricName("weightedRecall").evaluate(pred_df)
    f1w = mc.setMetricName("f1").evaluate(pred_df)
    cm = confusion_matrix(pred_df)
    prec = cm["tp"] / max(cm["tp"] + cm["fp"], 1)
    rec = cm["tp"] / max(cm["tp"] + cm["fn"], 1)
    f1 = 2 * prec * rec / max(prec + rec, 1e-12)
    return {"model": name, "accuracy": acc, "precision_churn": prec, "recall_churn": rec,
            "f1_churn": f1, "weighted_precision": wp, "weighted_recall": wr, "weighted_f1": f1w,
            "roc_auc": roc, "pr_auc": pr, "confusion_matrix": cm,
            "best_params": best_params, "fit_seconds": round(fit_seconds, 1)}


def main():
    config.ensure_dirs()
    spark = get_spark("TelcoChurnTraining")
    df = (spark.read.parquet(str(config.PROCESSED / "customers.parquet"))
          .withColumn("label", F.when(F.col("Churn") == "Yes", 1.0).otherwise(0.0)))

    # Stratified 80/20 split: sampleBy draws 80% of each class; the remainder is the test set.
    train = df.sampleBy("label", {0.0: config.TRAIN_FRACTION, 1.0: config.TRAIN_FRACTION}, seed=config.SEED)
    test = df.join(train.select("customerID"), "customerID", "left_anti")
    train.cache(); test.cache()
    n_train, n_test = train.count(), test.count()
    overlap = train.join(test, "customerID").count()
    print(f"[TRAIN] train={n_train} test={n_test} overlap={overlap}")
    assert overlap == 0 and n_train + n_test == config.EXPECTED_ROWS
    for nm, part in (("train", train), ("test", test)):
        rate = part.agg(F.avg("label")).first()[0]
        print(f"[TRAIN] {nm} churn prevalence = {rate:.4f}")
    majority = 1 - test.agg(F.avg("label")).first()[0]
    print(f"[TRAIN] majority-class baseline accuracy on test = {majority:.4f}")

    prep = build_prep_stages()
    ev_auc = BinaryClassificationEvaluator(metricName="areaUnderROC")
    models = {
        "LogisticRegression": (LogisticRegression(maxIter=100, labelCol="label", featuresCol="features"),
                               lambda c: ParamGridBuilder().addGrid(c.regParam, [0.001, 0.01, 0.1])
                               .addGrid(c.elasticNetParam, [0.0, 0.5]).build()),
        "DecisionTree": (DecisionTreeClassifier(maxDepth=5, impurity="gini", seed=config.SEED), None),
        "RandomForest": (RandomForestClassifier(numTrees=100, maxDepth=8, featureSubsetStrategy="sqrt",
                                                seed=config.SEED),
                         lambda c: ParamGridBuilder().addGrid(c.maxDepth, [5, 8])
                         .addGrid(c.numTrees, [50, 100]).build()),
        "GBTClassifier": (GBTClassifier(maxIter=50, maxDepth=5, seed=config.SEED), None),
        "LinearSVC": (LinearSVC(maxIter=100, regParam=0.1), None),
    }

    results, fitted = [], {}
    for name, (clf, grid_fn) in models.items():
```

```
        t0 = time.time()
        pipe = Pipeline(stages=prep + [clf])
        if grid_fn:
            cv = CrossValidator(estimator=pipe, estimatorParamMaps=grid_fn(clf), evaluator=ev_auc,
                                numFolds=3, seed=config.SEED, parallelism=2)
            model = cv.fit(train).bestModel
        else:
            model = pipe.fit(train)
        secs = time.time() - t0
        est = model.stages[-1]
        params = {p.name: est.getOrDefault(p) for p in est.extractParamMap()
                  if p.name in ("regParam", "elasticNetParam", "maxDepth", "numTrees", "maxIter")}
        res = evaluate(name, model.transform(test), params, secs)
        results.append(res); fitted[name] = model
        print(f"[TRAIN] {name:<20} acc={res['accuracy']:.4f} prec1={res['precision_churn']:.4f} "
              f"rec1={res['recall_churn']:.4f} f1={res['f1_churn']:.4f} AUC={res['roc_auc']:.4f} "
              f"PR-AUC={res['pr_auc']:.4f} ({secs:.1f}s) params={params}")

    # ---- Scoring model: Random Forest (provides calibrated-enough class probabilities) ----
    rf_model = fitted["RandomForest"]
    rf_model.write().overwrite().save(str(config.MODELS / "spark_rf_pipeline"))
    print(f"[TRAIN] PipelineModel saved to {config.MODELS / 'spark_rf_pipeline'}")

    # Feature importance with readable names from the assembler's ML attribute metadata.
    scored_meta = rf_model.transform(test.limit(5))
    attrs = scored_meta.schema["raw_features"].metadata["ml_attr"]["attrs"]
    pairs = sorted((a["idx"], a["name"]) for grp in attrs.values() for a in grp)
    names = [n for _, n in pairs]
    imp = rf_model.stages[-1].featureImportances.toArray()
    top = sorted(zip(names, imp), key=lambda t: -t[1])[:10]
    print("[TRAIN] Top-10 Random Forest feature importances:")
    for n, v in top:
        print(f"         {n:<42}{v:.4f}")
    plt.figure(figsize=(8, 5))
    plt.barh([n for n, _ in top][::-1], [float(v) for _, v in top][::-1], color="#e25a1c")
    plt.xlabel("Importance"); plt.title("Top 10 Feature Importances - Spark Random Forest")
    plt.tight_layout(); plt.savefig(config.REPORTS / "feature_importance.png", dpi=150); plt.close()

    # Score ALL customers; 'split' marks whether the row was seen in training (honest reporting).
    train_ids = train.select("customerID").withColumn("split", F.lit("train"))
    scored = (rf_model.transform(df)
              .withColumn("churn_probability", vector_to_array("probability")[1])
              .join(train_ids, "customerID", "left")
              .withColumn("split", F.coalesce("split", F.lit("test")))
              .withColumn("risk_level", F.when(F.col("churn_probability") >= config.HIGH_THRESHOLD, "High")
                          .when(F.col("churn_probability") >= config.MEDIUM_THRESHOLD, "Medium")
                          .otherwise("Low"))
              .withColumn("predicted_churn", F.col("prediction").cast("int"))
              .withColumn("revenue_at_risk", F.round(F.col("churn_probability") * F.col("MonthlyCharges"), 2))
              .select("customerID", "churn_probability", "predicted_churn", "risk_level", "Churn",
                      "MonthlyCharges", "tenure", "Contract", "InternetService", "revenue_at_risk", "split"))
    scored.write.mode("overwrite").parquet(str(config.PROCESSED / "predictions.parquet"))
    pdf = scored.toPandas()
    assert len(pdf) == config.EXPECTED_ROWS and pdf["churn_probability"].between(0, 1).all()
    pdf.sort_values("churn_probability", ascending=False).to_json(
        config.API_EXPORTS / "predictions.json", orient="records", indent=1)
    risk = pdf["risk_level"].value_counts().to_dict()
    print(f"[TRAIN] scored {len(pdf)} customers; risk distribution = {risk}")
    print(f"[TRAIN] total monthly revenue at risk = {pdf['revenue_at_risk'].sum():,.2f}")

    # Outputs for reports and the dashboard API
    evaluation = {"split": {"train": n_train, "test": n_test, "seed": config.SEED},
                  "majority_baseline_accuracy": majority, "scoring_model": "RandomForest",
                  "models": results,
                  "top_features": [{"feature": n, "importance": float(v)} for n, v in top]}
    config.write_json(config.REPORTS / "evaluation_summary.json", evaluation)
    pd.DataFrame([{k: v for k, v in r.items() if k not in ("confusion_matrix", "best_params")}
                  for r in results]).round(4).to_csv(config.REPORTS / "model_comparison.csv", index=False)

    summary_path = config.ANALYTICS / "dashboard_summary.json"
    summary = json.load(open(summary_path))
    summary.update({"riskBreakdown": risk, "highRiskCount": int(risk.get("High", 0)),
                    "avgChurnProbability": float(pdf["churn_probability"].mean()),
                    "monthlyRevenueAtRisk": float(pdf["revenue_at_risk"].sum())})
    config.write_json(summary_path, summary)
    spark.stop()


if __name__ == "__main__":
    main()
```

8.7 spark-pipeline/spark_clustering.py

```
"""Stage 4a - Unsupervised customer segmentation with K-Means (k=4) and PCA visualisation."""
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from pyspark.ml.clustering import KMeans
```

```
from pyspark.ml.evaluation import ClusteringEvaluator
from pyspark.ml.feature import PCA, StandardScaler, VectorAssembler
from pyspark.ml.functions import vector_to_array
from pyspark.sql import functions as F

import config
from spark_session import get_spark

FEATURES = ["tenure", "MonthlyCharges", "TotalCharges", "TotalServices", "ContractCode"]


def main():
    config.ensure_dirs()
    spark = get_spark("TelcoChurnClustering")
    df = (spark.read.parquet(str(config.PROCESSED / "customers.parquet"))
          .withColumn("ContractCode", F.when(F.col("Contract") == "Month-to-month", 0.0)
                      .when(F.col("Contract") == "One year", 1.0).otherwise(2.0)))
    assembled = VectorAssembler(inputCols=FEATURES, outputCol="raw").transform(df)
    scaled = (StandardScaler(inputCol="raw", outputCol="features", withMean=True, withStd=True)
              .fit(assembled).transform(assembled)).cache()
    evaluator = ClusteringEvaluator(featuresCol="features", predictionCol="prediction",
                                    metricName="silhouette", distanceMeasure="squaredEuclidean")

    # Model-selection sweep (supports the choice k = 4)
    sweep = []
    for k in range(2, 8):
        m = KMeans(k=k, seed=config.SEED, maxIter=50, initMode="k-means||").fit(scaled)
        sil = evaluator.evaluate(m.transform(scaled))
        sweep.append({"k": k, "silhouette": round(sil, 4), "wssse": round(m.summary.trainingCost, 1)})
        print(f"[KMEANS] k={k}  silhouette={sil:.4f}  WSSSE={m.summary.trainingCost:,.1f}")

    model = KMeans(k=4, seed=config.SEED, maxIter=50, initMode="k-means||").fit(scaled)
    clustered = model.transform(scaled).withColumnRenamed("prediction", "cluster")
    silhouette = evaluator.evaluate(model.transform(scaled))
    print(f"[KMEANS] final k=4 silhouette = {silhouette:.4f}")

    profile = (clustered.groupBy("cluster").agg(
        F.count("*").alias("customers"), F.round(F.avg("tenure"), 1).alias("avg_tenure"),
        F.round(F.avg("MonthlyCharges"), 2).alias("avg_monthly"),
        F.round(F.avg("TotalServices"), 2).alias("avg_services"),
        F.round(F.avg("ContractCode"), 2).alias("avg_contract_code"),
        F.round(100 * F.avg(F.when(F.col("Churn") == "Yes", 1.0).otherwise(0.0)), 2).alias("churn_rate_pct"))
        .orderBy("cluster"))
    profile.show(truncate=False)

    clustered.select("customerID", "cluster", *FEATURES, "Churn").write.mode("overwrite").parquet(
        str(config.ANALYTICS / "customer_segments.parquet"))
    config.write_json(config.REPORTS / "clustering_summary.json",
                      {"k": 4, "silhouette": silhouette, "sweep": sweep,
                       "profiles": [r.asDict() for r in profile.collect()]})

    pca = PCA(k=2, inputCol="features", outputCol="pca").fit(clustered)
    pts = (pca.transform(clustered).select("cluster", vector_to_array("pca").alias("p"))
           .select("cluster", F.col("p")[0].alias("x"), F.col("p")[1].alias("y")).toPandas())
    print("[KMEANS] PCA explained variance (2 comps):", [round(float(v), 3) for v in pca.explainedVariance])
    plt.figure(figsize=(7, 5.5))
    for c, g in pts.groupby("cluster"):
        plt.scatter(g.x, g.y, s=6, alpha=0.6, label=f"Cluster {c}")
    plt.legend(); plt.xlabel("PC1"); plt.ylabel("PC2"); plt.title("K-Means Customer Segments (PCA projection)")
    plt.tight_layout(); plt.savefig(config.REPORTS / "customer_clusters.png", dpi=150); plt.close()
    spark.stop()


if __name__ == "__main__":
    main()
```

8.8 spark-pipeline/spark_regression.py

```
"""Stage 4b - Linear Regression estimating MonthlyCharges from subscribed services."""
from pyspark.ml import Pipeline
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml.feature import OneHotEncoder, StringIndexer, VectorAssembler
from pyspark.ml.regression import LinearRegression
from pyspark.sql import functions as F

import config
from spark_session import get_spark

# Yes/No service flags. "No phone service" / "No internet service" levels are redundant with
# PhoneService / InternetService, so using full one-hot encodings would make the design matrix singular.
FLAG_COLS = ["PhoneService", "MultipleLines", "OnlineSecurity", "OnlineBackup",
             "DeviceProtection", "TechSupport", "StreamingTV", "StreamingMovies"]


def feature_names(df, col):
    attrs = df.schema[col].metadata["ml_attr"]["attrs"]
    return [n for _, n in sorted((a["idx"], a["name"]) for g in attrs.values() for a in g)]
```

```
def main():
    config.ensure_dirs()
    spark = get_spark("TelcoChurnRegression")
    df = spark.read.parquet(str(config.PROCESSED / "customers.parquet"))
    df = df.select("*", *[(F.col(c) == "Yes").cast("double").alias(f"{c}_yes") for c in FLAG_COLS])
    train, test = df.randomSplit([0.8, 0.2], seed=config.SEED)   # no TotalCharges-derived inputs used

    prep = [StringIndexer(inputCol="InternetService", outputCol="InternetService_idx"),
            OneHotEncoder(inputCol="InternetService_idx", outputCol="InternetService_ohe"),  # 'No' = reference
            VectorAssembler(inputCols=["InternetService_ohe"] + [f"{c}_yes" for c in FLAG_COLS],
                            outputCol="features")]
    evaluator = RegressionEvaluator(labelCol="MonthlyCharges", predictionCol="prediction")
    out = {}

    # (a) Specification model: elastic-net regularised regression
    enet = LinearRegression(featuresCol="features", labelCol="MonthlyCharges",
                            maxIter=100, regParam=0.3, elasticNetParam=0.8)
    # (b) Un-regularised OLS via the normal equation (exposes p-values and standard errors)
    ols = LinearRegression(featuresCol="features", labelCol="MonthlyCharges",
                           regParam=0.0, solver="normal")
    for tag, est in (("elastic_net", enet), ("ols", ols)):
        model = Pipeline(stages=prep + [est]).fit(train)
        pred = model.transform(test)
        metrics = {m: evaluator.setMetricName(m).evaluate(pred) for m in ("r2", "rmse", "mae")}
        print(f"[REGRESSION] {tag:<12} R2={metrics['r2']:.4f}  RMSE={metrics['rmse']:.4f}
MAE={metrics['mae']:.4f}")
        lr = model.stages[-1]
        names = feature_names(pred, "features")
        out[tag] = {"test_metrics": metrics, "intercept": lr.intercept,
                    "coefficients": dict(zip(names, [float(c) for c in lr.coefficients]))}
        if tag == "ols":
            s = lr.summary
            out[tag]["train_r2"] = s.r2
            out[tag]["p_values"] = dict(zip(names + ["intercept"], [float(p) for p in s.pValues]))
            print("[REGRESSION] OLS largest effects (coefficient, p-value):")
            top = sorted(out[tag]["coefficients"].items(), key=lambda t: -abs(t[1]))[:6]
            for n, c in top:
                print(f"              {n:<38}{c:>9.3f}   p={out[tag]['p_values'][n]:.4g}")
    config.write_json(config.REPORTS / "regression_summary.json", out)
    spark.stop()


if __name__ == "__main__":
    main()
```

8.9 scala-companion/build.sbt

```
name := "telco-churn-scala"
version := "1.0"
scalaVersion := "2.12.18"

libraryDependencies ++= Seq(
  "org.apache.spark" %% "spark-core" % "3.5.3" % "provided",
  "org.apache.spark" %% "spark-sql"  % "3.5.3" % "provided"
)

// 'sbt run' needs the "provided" Spark jars on the classpath:
Compile / run := Defaults.runTask(Compile / fullClasspath, Compile / run / mainClass, Compile / run /
runner).evaluated
```

8.10 scala-companion/src/main/scala/telco/churn/ChurnApp.scala

```
package telco.churn

import org.apache.spark.sql.{Dataset, SparkSession}

/** Strongly-typed record: one row of the cleaned customers.parquet mart. */
case class Customer(customerID: String, tenure: Int, MonthlyCharges: Double, TotalCharges: Double,
                    Contract: String, InternetService: String, TotalServices: Int, Churn: String)

object ChurnApp {
  def main(args: Array[String]): Unit = {
    val path = if (args.nonEmpty) args(0) else "data/processed/customers.parquet"

    val spark = SparkSession.builder().appName("TelcoChurnScala").master("local[*]").getOrCreate()
    spark.sparkContext.setLogLevel("ERROR")
    import spark.implicits._

    // Type inference: the compiler infers Dataset[Customer]; the case-class encoder checks columns/types.
    val customers: Dataset[Customer] = spark.read.parquet(path)
      .select("customerID", "tenure", "MonthlyCharges", "TotalCharges",
              "Contract", "InternetService", "TotalServices", "Churn")
      .as[Customer]

    println(s"[SCALA] Typed Dataset[Customer] loaded: ${customers.count()} records")
```

```
    // Typed, functional Dataset API (compile-time checked lambdas instead of column strings)
    val churned = customers.filter(_.Churn == "Yes").count()
    println(f"[SCALA] churned = $churned, churn rate = ${100.0 * churned / customers.count()}%.2f%%")

    val byContract = customers.groupByKey(_.Contract).mapGroups { (contract, rows) =>
      val list = rows.toList
      (contract, list.size, 100.0 * list.count(_.Churn == "Yes") / list.size)
    }.collect().sortBy(-_._3)
    byContract.foreach { case (c, n, rate) => println(f"[SCALA]   $c%-15s customers=$n%5d churn=$rate%6.2f%%") }

    // Hand over to the pure-Scala collection and lazy-evaluation demonstrations
    val all: List[Customer] = customers.collect().toList      // immutable List
    CollectionsDemo.run(all)
    StreamSimulation.run()
    spark.stop()
  }
}
```

8.11 scala-companion/src/main/scala/telco/churn/CollectionsDemo.scala

```
package telco.churn

import scala.collection.mutable.ArrayBuffer

object CollectionsDemo {

  /** Pattern matching on a tuple: business risk segment from contract + internet type. */
  def segment(c: Customer): String = (c.Contract, c.InternetService) match {
    case ("Month-to-month", "Fiber optic") => "Critical"
    case ("Month-to-month", _)             => "Elevated"
    case (_, "Fiber optic")                => "Watch"
    case _                                 => "Stable"
  }

  def run(customers: List[Customer]): Unit = {
    // --- Immutable collections & higher-order functions ---
    val (churned, retained) = customers.partition(_.Churn == "Yes")
    val avgTenureChurned  = churned.map(_.tenure).sum.toDouble / churned.size
    val avgTenureRetained = retained.map(_.tenure).sum.toDouble / retained.size
    println(f"[COLLECTIONS] avg tenure  churned=$avgTenureChurned%.2f  retained=$avgTenureRetained%.2f months")

    // Closure capturing a val + function literal stored in a val
    val threshold = customers.map(_.MonthlyCharges).sum / customers.size
    val isHighSpender = (c: Customer) => c.MonthlyCharges > threshold
    val highSpenderChurn = customers.filter(isHighSpender).count(_.Churn == "Yes")
    println(f"[COLLECTIONS] mean monthly charge = $threshold%.2f ; churned high-spenders = $highSpenderChurn")

    // groupBy -> immutable Map[String, List[Customer]], then foldLeft per group
    val bySegment: Map[String, List[Customer]] = customers.groupBy(segment)
    val report = bySegment.map { case (seg, xs) =>
      val churnCount = xs.foldLeft(0)((acc, c) => if (c.Churn == "Yes") acc + 1 else acc)
      (seg, xs.size, 100.0 * churnCount / xs.size)
    }.toList.sortBy(-_._3)
    report.foreach { case (s, n, r) => println(f"[COLLECTIONS]   segment=$s%-9s customers=$n%5d
churn=$r%6.2f%%") }

    // Immutable Set / Map: "updating" returns a NEW collection, the original is unchanged
    val internetTypes = customers.map(_.InternetService).toSet
    val extended = internetTypes + "Satellite"
    println(s"[COLLECTIONS] immutable Set original=${internetTypes.size} extended=${extended.size}")

    // Composable rule-based score: List of functions folded over a customer
    val rules: List[Customer => Double] = List(
      c => if (c.Contract == "Month-to-month") 0.40 else 0.0,
      c => if (c.tenure < 12) 0.30 else 0.0,
      c => if (c.InternetService == "Fiber optic") 0.20 else 0.0,
      c => if (c.TotalServices <= 1) 0.10 else 0.0)
    val score = (c: Customer) => rules.foldLeft(0.0)((acc, r) => acc + r(c))
    val flagged = customers.filter(c => score(c) >= 0.8)
    println(f"[COLLECTIONS] rule score >= 0.8 flags ${flagged.size} customers, " +
            f"${100.0 * flagged.count(_.Churn == "Yes") / flagged.size}%.2f%% of them churned")

    // Mutable vs immutable: ArrayBuffer is modified in place (not thread-safe)
    val buf = ArrayBuffer[String]()
    buf += "a"; buf += "b"
    println(s"[COLLECTIONS] mutable ArrayBuffer after two appends = $buf")
  }
}
```

8.12 scala-companion/src/main/scala/telco/churn/StreamSimulation.scala

```
package telco.churn

import scala.util.Random

/** Lazy evaluation: an infinite call-event stream where elements are computed only on demand. */
object StreamSimulation {
```

```
  case class CallEvent(id: Int, durationSec: Int, dropped: Boolean)

  def run(): Unit = {
    val rnd = new Random(42)
    var generated = 0                       // counts how many events are really materialised

    val events: Stream[CallEvent] = Stream.from(1).map { i =>
      generated += 1
      CallEvent(i, rnd.nextInt(600), rnd.nextDouble() < 0.15)
    }
    println(s"[STREAM] infinite stream defined, but head-only evaluation so far: generated = $generated")

    val firstDropped = events.filter(_.dropped).take(5).toList    // forces only what is needed
    firstDropped.foreach(e => println(s"[STREAM]   dropped call -> $e"))
    println(s"[STREAM] events materialised to find 5 dropped calls = $generated")
  }
}
```

8.13 dashboard-api/package.json

```
{
  "name": "telco-churn-spark-api",
  "version": "1.0.0",
  "description": "File-based analytical API serving Spark-generated marts (no database)",
  "main": "server.js",
  "scripts": { "start": "node server.js" },
  "dependencies": { "cors": "^2.8.5", "express": "^5.1.0" }
}
```

8.14 dashboard-api/.env

```
PORT=4000
DATA_DIR=../data/processed
REPORTS_DIR=../reports
```

8.15 dashboard-api/server.js

```
// Decoupled analytical API: serves Spark-generated JSON marts from memory (no database, no Spark JVM).
const fs = require("fs");
const path = require("path");
const express = require("express");
const cors = require("cors");

// minimal .env loader (avoids an extra dependency)
if (fs.existsSync(path.join(__dirname, ".env"))) {
  for (const line of fs.readFileSync(path.join(__dirname, ".env"), "utf8").split(/\r?\n/)) {
    const m = line.match(/^\s*([A-Z_]+)\s*=\s*(.*)\s*$/);
    if (m && !(m[1] in process.env)) process.env[m[1]] = m[2];
  }
}
const PORT = process.env.PORT || 4000;
const DATA_DIR = path.resolve(__dirname, process.env.DATA_DIR || "../data/processed");
const REPORTS_DIR = path.resolve(__dirname, process.env.REPORTS_DIR || "../reports");
const FILES = {
  summary: path.join(DATA_DIR, "analytics", "dashboard_summary.json"),
  customers: path.join(DATA_DIR, "api_exports", "customers.json"),
  predictions: path.join(DATA_DIR, "api_exports", "predictions.json"),
  model: path.join(REPORTS_DIR, "evaluation_summary.json"),
};

const store = {};
function load(key) {
  try {
    store[key] = JSON.parse(fs.readFileSync(FILES[key], "utf8"));
    console.log(`[cache] loaded ${key} (${Array.isArray(store[key]) ? store[key].length + " records" :
"object"})`);
  } catch (err) {
    console.warn(`[cache] ${key} not available: ${err.message}`);
    store[key] = null;
  }
}
// load on boot + hot-reload when Spark rewrites a file (debounced)
for (const key of Object.keys(FILES)) {
  load(key);
  const timers = {};
  if (fs.existsSync(FILES[key])) {
    fs.watch(FILES[key], () => { clearTimeout(timers[key]); timers[key] = setTimeout(() => load(key), 500); });
  }
}

const app = express();
app.use(cors());
const paginate = (rows, page, limit) => {
  const p = Math.max(parseInt(page) || 1, 1), l = Math.min(Math.max(parseInt(limit) || 20, 1), 200);
  return { page: p, limit: l, total: rows.length, pages: Math.ceil(rows.length / l),
           data: rows.slice((p - 1) * l, p * l) };
```

```
};
const needs = (key, res) => store[key] ? true : (res.status(503).json({ error: `Spark mart '${key}' not loaded`
}), false);

app.get("/api/health", (req, res) => {
  const ready = Object.fromEntries(Object.keys(FILES).map(k => [k, store[k] !== null]));
  res.json({ status: Object.values(ready).every(Boolean) ? "ok" : "degraded", engine: "Apache Spark data mart",
ready });
});

app.get("/api/dashboard", (req, res) => {
  if (!needs("summary", res)) return;
  const s = store.summary;
  res.json({ totalCustomers: s.kpis.total_customers, churned: s.kpis.churned, churnRate: s.kpis.churn_rate_pct,
             highRisk: s.highRiskCount, avgChurnProbability: s.avgChurnProbability,
             monthlyRevenueAtRisk: s.monthlyRevenueAtRisk, churnByContract: s.contractChurn,
             churnByInternet: s.internetChurn, riskBreakdown: s.riskBreakdown });
});

app.get("/api/customers", (req, res) => {
  if (!needs("customers", res)) return;
  const { contract, churn, tenureMin, tenureMax, page, limit } = req.query;
  let rows = store.customers.filter(c =>
    (!contract || c.Contract === contract) && (!churn || c.Churn === churn) &&
    (tenureMin === undefined || c.tenure >= Number(tenureMin)) &&
    (tenureMax === undefined || c.tenure <= Number(tenureMax)));
  const avg = k => rows.length ? +(rows.reduce((a, c) => a + c[k], 0) / rows.length).toFixed(2) : 0;
  res.json({ ...paginate(rows, page, limit),
             summary: { avgTenure: avg("tenure"), avgMonthly: avg("MonthlyCharges"), avgTotal:
avg("TotalCharges") } });
});

app.get("/api/predictions", (req, res) => {
  if (!needs("predictions", res)) return;
  const { search, risk_level, sort, page, limit } = req.query;
  let rows = store.predictions.filter(p =>
    (!risk_level || p.risk_level === risk_level) &&
    (!search || new RegExp(search.replace(/[.*+?^${}()|[\]\\]/g, "\\$&"), "i").test(p.customerID)));
  rows = [...rows].sort((a, b) => sort === "asc" ? a.churn_probability - b.churn_probability
                                                 : b.churn_probability - a.churn_probability);
  res.json(paginate(rows, page, limit));
});

app.get("/api/model", (req, res) => store.model ? res.json(store.model) : needs("model", res));
app.get("/api/feature-importance.png", (req, res) => res.sendFile(path.join(REPORTS_DIR,
"feature_importance.png")));

app.listen(PORT, () => console.log(`Telco Spark API listening on http://localhost:${PORT}`));
```

9. Execution Screenshots

The figures below show the console output captured while the project was executed, stage by stage, on the real 7,043-record dataset.

Environment check

```
$ java -version
openjdk version "21.0.10" 2026-01-20
OpenJDK Runtime Environment (build 21.0.10+7-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.10+7-Ubuntu-124.04, mixed mode, sharing)
$ python --version
Python 3.12.3
$ python -c "import pyspark; print(pyspark.__version__)"
3.5.3
$ node --version
v22.22.2
```

Figure 9.1 – Software environment (Java, Python, PySpark and Node.js versions)

Project structure and generated artifacts

```
$ ls spark-pipeline scala-companion/src/main/scala/telco/churn dashboard-api
spark-pipeline:
  config.py  spark_session.py  spark_ingest.py  spark_analytics.py
  spark_train.py  spark_clustering.py  spark_regression.py

scala-companion/src/main/scala/telco/churn:
  ChurnApp.scala  CollectionsDemo.scala  StreamSimulation.scala

dashboard-api:
  package.json  .env  server.js

$ ls data/raw data/processed data/processed/analytics data/processed/api_exports models reports
data/raw:
  WA_Fn-UseC_-Telco-Customer-Churn.csv
data/processed:
  analytics  api_exports  customers.parquet  customers_csv  predictions.parquet
data/processed/analytics:
  contract_churn.parquet  customer_segments.parquet  dashboard_summary.json
data/processed/api_exports:
  customers.json  predictions.json
models:
  spark_rf_pipeline
reports:
  clustering_summary.json  correlation_heatmap.png  correlation_matrix.json  customer_clusters.png
    evaluation_summary.json  feature_importance.png  model_comparison.csv  regression_summary.json
```

Figure 9.2 – Project structure and the Parquet / JSON / model / report artefacts generated by Spark

spark_ingest.py

```
$ python spark-pipeline/spark_ingest.py
[INGEST] rows=7043  columns=21  duplicate customerIDs=0
[INGEST] blank/null TotalCharges before imputation: 11
root
 |-- customerID: string (nullable = true)
 |-- gender: string (nullable = true)
  ... (schema listing: 27 columns incl. 6 engineered features) ...
[INGEST] wrote Parquet, CSV and JSON marts (7043 records)
```

Figure 9.3 – Stage 1: ingestion – row/column validation, 11 blank TotalCharges imputed, Parquet mart written

spark_analytics.py - Spark SQL aggregations

```
$ python spark-pipeline/spark_analytics.py
[ANALYTICS] KPIs: {'total_customers': 7043, 'churned': 1869, 'churn_rate_pct': 26.54, 'avg_tenure': 32.37,
    'avg_monthly': 64.76, 'avg_total': 2279.73}
[ANALYTICS] Churn rate by Contract
+--------------+---------+-------+--------------+----------+-----------+
|segment       |customers|churned|churn_rate_pct|avg_tenure|avg_monthly|
+--------------+---------+-------+--------------+----------+-----------+
|Month-to-month|3875     |1655   |42.71         |18.04     |66.4       |
|One year      |1473     |166    |11.27         |42.04     |65.05      |
|Two year      |1695     |48     |2.83          |56.74     |60.77      |
+--------------+---------+-------+--------------+----------+-----------+
[ANALYTICS] Churn rate by InternetService
+-----------+---------+-------+--------------+----------+-----------+
|segment    |customers|churned|churn_rate_pct|avg_tenure|avg_monthly|
+-----------+---------+-------+--------------+----------+-----------+
|Fiber optic|3096     |1297   |41.89         |32.92     |91.5       |
|DSL        |2421     |459    |18.96         |32.82     |58.1       |
|No         |1526     |113    |7.41          |30.55     |21.08      |
+-----------+---------+-------+--------------+----------+-----------+
[ANALYTICS] Churn rate by TenureGroup
+-------+---------+-------+--------------+----------+-----------+
|segment|customers|churned|churn_rate_pct|avg_tenure|avg_monthly|
+-------+---------+-------+--------------+----------+-----------+
|0-12   |2186     |1037   |47.44         |4.73      |56.1       |
|13-24  |1024     |294    |28.71         |18.35     |61.36      |
|25-48  |1594     |325    |20.39         |36.19     |65.93      |
|49-72  |2239     |213    |9.51          |63.05     |73.95      |
+-------+---------+-------+--------------+----------+-----------+
[ANALYTICS] Churn rate by PaymentMethod
+------------------------+---------+-------+--------------+----------+-----------+
|segment                 |customers|churned|churn_rate_pct|avg_tenure|avg_monthly|
+------------------------+---------+-------+--------------+----------+-----------+
|Electronic check        |2365     |1071   |45.29         |25.17     |76.26      |
|Mailed check            |1612     |308    |19.11         |21.83     |43.92      |
|Bank transfer (automatic)|1544     |258    |16.71         |43.66     |67.19      |
|Credit card (automatic) |1522     |232    |15.24         |43.27     |66.51      |
+------------------------+---------+-------+--------------+----------+-----------+
```

Figure 9.4 – Stage 2: Spark SQL KPIs and churn rate by Contract, InternetService, TenureGroup and PaymentMethod

df.explain(True)

```
$ ... .explain(True)   # Physical Plan (Catalyst + Tungsten)
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Sort [churn_rate_pct#567 DESC NULLS LAST], true, 0
   +- Exchange rangepartitioning(churn_rate_pct#567 DESC NULLS LAST, 8), ENSURE_REQUIREMENTS,
      +- HashAggregate(keys=[Contract#15], functions=[count(1), sum(CASE WHEN (Churn#20 = Yes) THEN 1 ELSE 0
    END), avg(UnscaledValue(CASE WHEN (Churn#20 = Yes) THEN 1.0 ELSE 0.0 END)), avg(tenure# ...
         +- Exchange hashpartitioning(Contract#15, 8), ENSURE_REQUIREMENTS,
            +- HashAggregate(keys=[Contract#15], functions=[partial_count(1), partial_sum(CASE WHEN (Churn#20 =
    Yes) THEN 1 ELSE 0 END), partial_avg(UnscaledValue(CASE WHEN (Churn#20 = Yes) THEN 1 ...
               +- FileScan parquet [tenure#5,Contract#15,MonthlyCharges#18,Churn#20] Batched: true, DataFilters:
     [], Format: Parquet, Location: InMemoryFileIndex(1 paths)[file:data/processed/custo ...
```

Figure 9.5 – Catalyst physical plan from df.explain(True): Parquet scan with column pruning and two-phase HashAggregate

Caching benchmark and Pearson correlation

```
[ANALYTICS] 5 repeated aggregations - uncached: 1.743s, cached: 1.394s
[ANALYTICS] Pearson correlation matrix:
                          tenure  MonthlyCharges    TotalCharges AvgMonthlySpend ChargeDifferenc
tenure                     1.000           0.248           0.826           0.414          -0.578
MonthlyCharges             0.248           1.000           0.651           0.956           0.095
TotalCharges               0.826           0.651           1.000           0.765          -0.421
AvgMonthlySpend            0.414           0.956           0.765           1.000          -0.201
ChargeDifference          -0.578           0.095          -0.421          -0.201           1.000
```

Figure 9.6 – In-memory caching benchmark and Pearson correlation matrix

spark_train.py

```
$ python spark-pipeline/spark_train.py
[TRAIN] train=5698 test=1345 overlap=0
[TRAIN] train churn prevalence = 0.2629
[TRAIN] test churn prevalence = 0.2758
[TRAIN] majority-class baseline accuracy on test = 0.7242
[TRAIN] LogisticRegression   acc=0.8089 prec1=0.7375 rec1=0.4771 f1=0.5794 AUC=0.8369 PR-AUC=0.6665 (91.4s)
[TRAIN] DecisionTree         acc=0.7755 prec1=0.6283 rec1=0.4555 f1=0.5281 AUC=0.7154 PR-AUC=0.4600 (3.4s)
[TRAIN] RandomForest         acc=0.7888 prec1=0.6693 rec1=0.4636 f1=0.5478 AUC=0.8314 PR-AUC=0.6506 (45.4s)
[TRAIN] GBTClassifier        acc=0.7755 prec1=0.6177 rec1=0.4879 f1=0.5452 AUC=0.8293 PR-AUC=0.6278 (14.2s)
[TRAIN] LinearSVC            acc=0.7903 prec1=0.7070 rec1=0.4097 f1=0.5188 AUC=0.8275 PR-AUC=0.6517 (5.3s)
[TRAIN] PipelineModel saved to models/spark_rf_pipeline
[TRAIN] Top-10 Random Forest feature importances:
         Contract_ohe_Month-to-month               0.1074
         ChargeDifference                          0.1038
         TenureRatio                               0.0814
         tenure                                    0.0745
         TechSupport_ohe_No                        0.0662
         InternetService_ohe_Fiber optic           0.0626
         OnlineSecurity_ohe_No                     0.0496
         TotalCharges                              0.0424
         MonthlyCharges                            0.0419
         Contract_ohe_Two year                     0.0337
[TRAIN] scored 7043 customers; risk distribution = {'Low': 4536, 'Medium': 1651, 'High': 856}
[TRAIN] total monthly revenue at risk = 136,244.44
```

Figure 9.7 – Stage 3: Spark ML training, 3-fold cross-validation, evaluation, feature importance and scoring

spark_clustering.py

```
$ python spark-pipeline/spark_clustering.py
[KMEANS] k=2  silhouette=0.5991  WSSSE=19,136.3
[KMEANS] k=3  silhouette=0.5900  WSSSE=13,379.8
[KMEANS] k=4  silhouette=0.5252  WSSSE=10,263.4
[KMEANS] k=5  silhouette=0.4821  WSSSE=9,087.3
[KMEANS] k=6  silhouette=0.5011  WSSSE=7,539.0
[KMEANS] k=7  silhouette=0.4777  WSSSE=6,969.0
[KMEANS] final k=4 silhouette = 0.5252
+-------+---------+----------+-----------+------------+-----------------+--------------+
|cluster|customers|avg_tenure|avg_monthly|avg_services|avg_contract_code|churn_rate_pct|
+-------+---------+----------+-----------+------------+-----------------+--------------+
|0      |2115     |8.2       |46.08      |0.53        |0.08             |39.67         |
|1      |2116     |25.6      |81.68      |2.58        |0.19             |38.56         |
|2      |1066     |45.8      |25.55      |0.34        |1.68             |1.41          |
|3      |1746     |61.7      |90.83      |4.25        |1.43             |11.4          |
+-------+---------+----------+-----------+------------+-----------------+--------------+
[KMEANS] PCA explained variance (2 comps): [0.619, 0.262]
```

Figure 9.8 – Stage 4a: K-Means silhouette sweep, final k = 4 cluster profile and PCA variance

spark_regression.py

```
$ python spark-pipeline/spark_regression.py
[REGRESSION] elastic_net  R2=0.9983  RMSE=1.2500  MAE=1.0193
[REGRESSION] ols          R2=0.9989  RMSE=1.0275  MAE=0.7891
[REGRESSION] OLS largest effects (coefficient, p-value):
             InternetService_ohe_Fiber optic          50.047   p=0
             InternetService_ohe_DSL                  25.073   p=0
             PhoneService_yes                         20.063   p=0
             StreamingMovies_yes                       9.976   p=0
             StreamingTV_yes                           9.930   p=0
             DeviceProtection_yes                      5.042   p=0
```

Figure 9.9 – Stage 4b: Linear Regression (elastic net vs OLS) metrics and largest coefficients



Figure 9.10 – Scala companion module executed with spark-submit

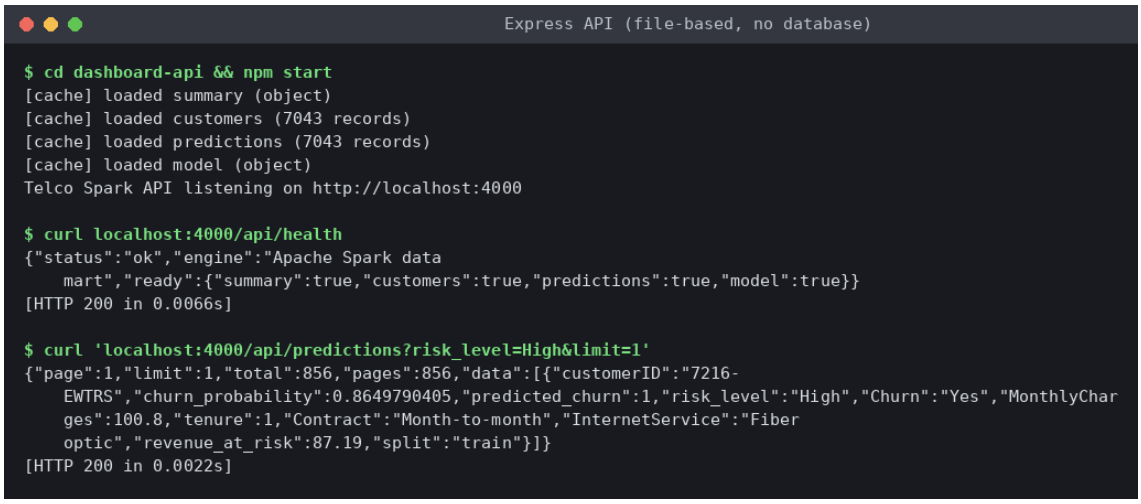


Figure 9.11 – Express API started from the Spark data mart and queried with curl

10. Output / Result Screenshots

Table 10.1 – Classification results on the held-out test set (n = 1,345; churn prevalence 27.58%)

Model	Accuracy	Precision (churn)	Recall (churn)	F1 (churn)	ROC-AUC	PR-AUC
LogisticRegression	0.8089	0.7375	0.4771	0.5794	0.8369	0.6665
DecisionTree	0.7755	0.6283	0.4555	0.5281	0.7154	0.4600
RandomForest	0.7888	0.6693	0.4636	0.5478	0.8314	0.6506
GBTClassifier	0.7755	0.6177	0.4879	0.5452	0.8293	0.6278
LinearSVC	0.7903	0.7070	0.4097	0.5188	0.8275	0.6517

Majority-class baseline accuracy on the same test set: **72.42%**.

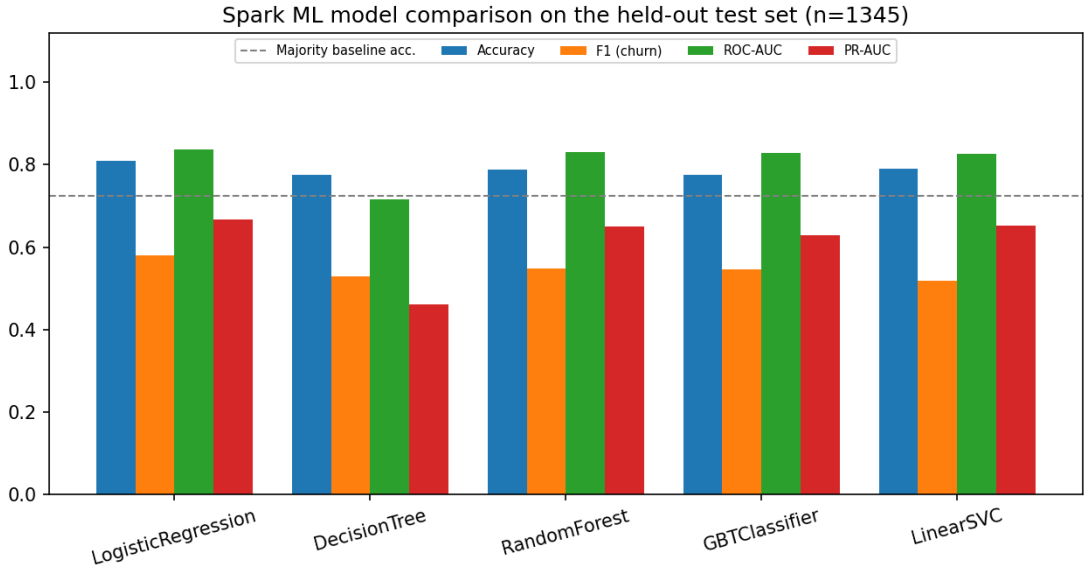


Figure 10.1 – Model comparison: accuracy, F1, ROC-AUC and PR-AUC against the majority-class baseline

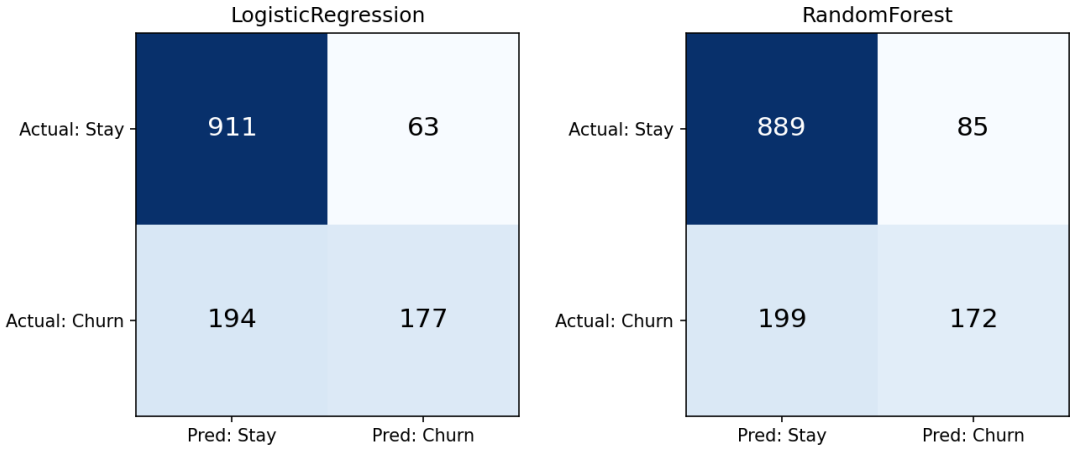


Figure 10.2 – Confusion matrices of Logistic Regression and Random Forest on the test set

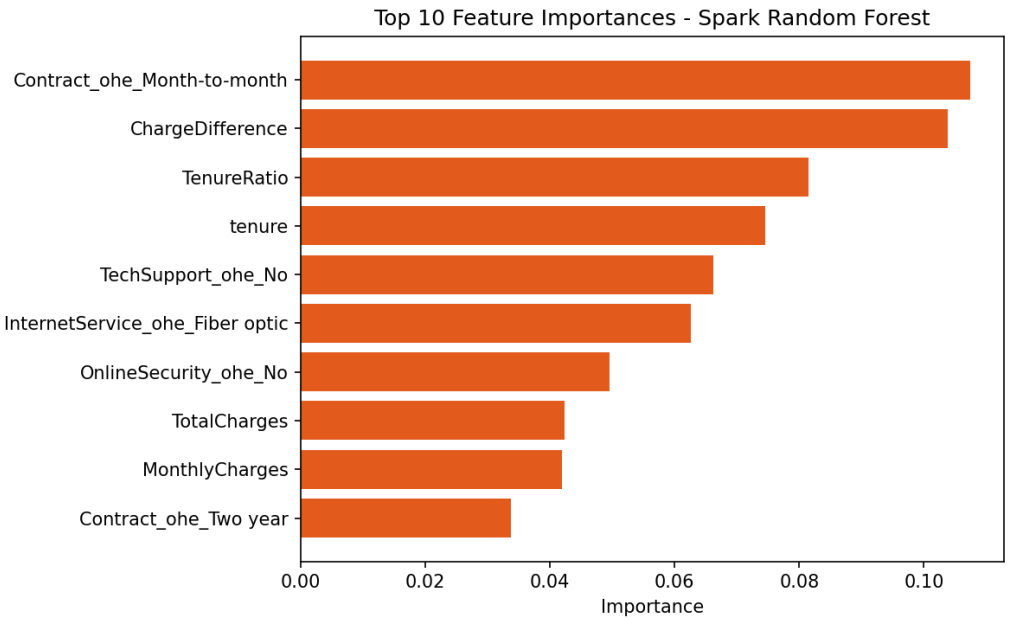


Figure 10.3 – Top 10 feature importances of the Spark Random Forest

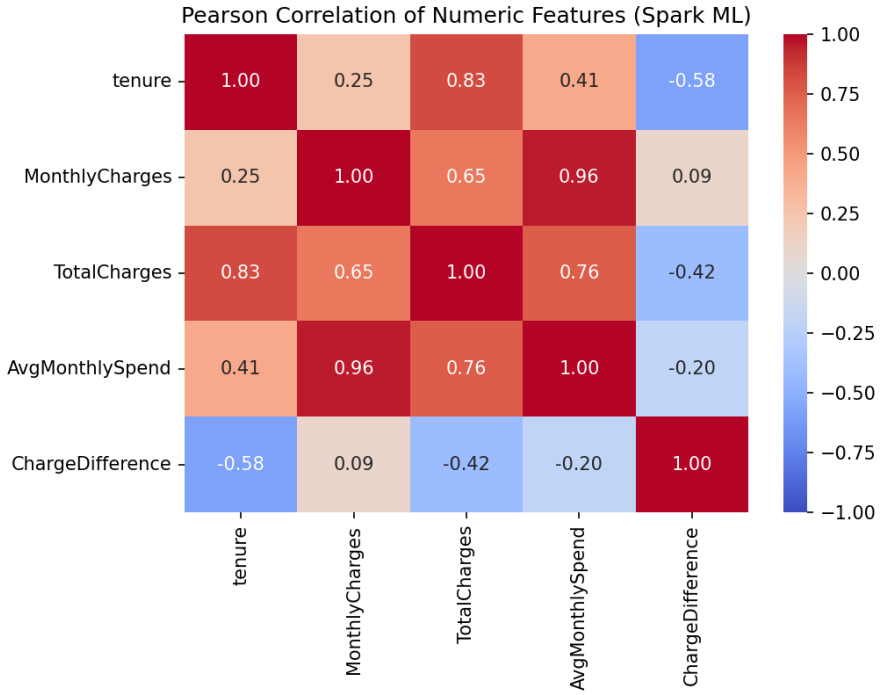


Figure 10.4 – Pearson correlation heatmap of numeric features (Spark ML Correlation.corr)



Figure 10.5 – K-Means customer segments (k = 4) projected with PCA

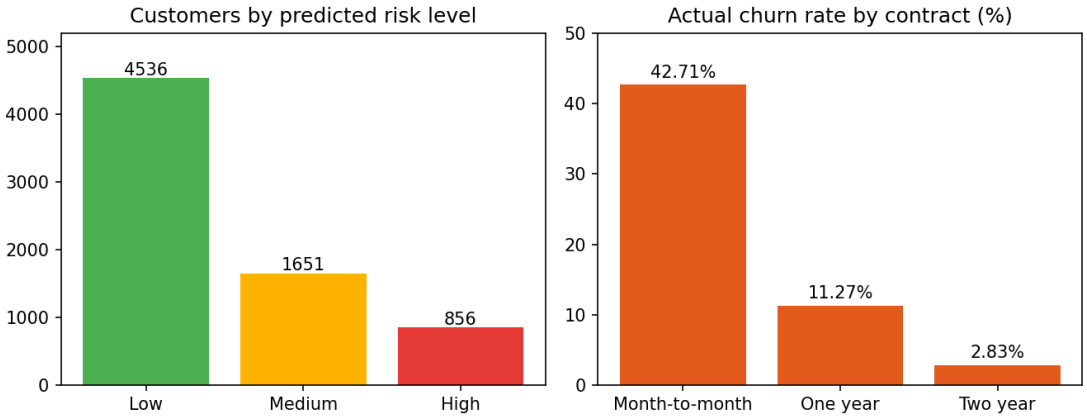


Figure 10.6 – Customers by predicted risk level and actual churn rate by contract

11. Result Analysis

Churn drivers from Spark SQL analytics

Overall, 1,869 of 7,043 customers (26.54%) churned. Contract type is the strongest single driver: churn is **42.71%** for month-to-month customers, 11.27% for one-year and only **2.83%** for two-year contracts. Fiber-optic customers churn at 41.89% compared with 18.96% for DSL and 7.41% for customers without Internet service. Churn falls steadily with tenure – 47.44% in the first 12 months against 9.51% after 48 months – and customers paying by electronic check churn at 45.29%, more than double the 15–19% of the other payment methods. Fiber customers also pay the most (average 91.50 per month versus 58.10 for DSL), which indicates a price-sensitive, high-value segment.

In the correlation matrix, tenure and TotalCharges are strongly related (0.826), and MonthlyCharges and AvgMonthlySpend are almost redundant (0.956); such collinearity inflates coefficient variance in linear models, and correlation here indicates association, not causation.

Classification performance

On the realistic, unresampled test set the models reach **77.6%–80.9% accuracy** against a 72.42% majority-class baseline. Logistic Regression is the best overall (accuracy 80.89%, ROC-AUC 0.8369, PR-AUC 0.6665, churn-class F1 0.5794), followed closely by Random Forest (ROC-AUC 0.8314), GBT (0.8293) and LinearSVC (0.8275). The single Decision Tree is weakest (ROC-AUC 0.7154). The near-tie between a linear model and tree ensembles suggests that most of the signal in this dataset is captured by a few additive effects, so extra model complexity brings little gain.

Recall for the churn class is only 41%–49% at the default 0.5 decision threshold: the models miss more than half of the customers who leave, while precision is 62%–74%. This is the typical effect of class imbalance, and it is why the project also reports ranking quality (ROC-AUC, PR-AUC) and uses probability-based risk tiers rather than a single hard label. These figures are intentionally much lower than the 96–97% accuracy of an earlier SMOTEENN-based prototype, because that figure was measured on an artificially resampled and boundary-cleaned test set and is not comparable with an honest hold-out evaluation.

The Random Forest's most important features are month-to-month contract (0.107), ChargeDifference (0.104), TenureRatio (0.081), tenure (0.075), absence of TechSupport (0.066), fiber-optic Internet (0.063) and absence of OnlineSecurity (0.050), matching the SQL findings. Note that tenure and TenureRatio carry the same information (TenureRatio = tenure /

72), so their importance is shared, and ChargeDifference is derived from TotalCharges and tenure and therefore partly encodes tenure.

Risk stratification

Scoring all 7,043 customers gives **856 High-risk** ($P \geq 0.60$), 1,651 Medium-risk and 4,536 Low-risk customers, with an expected monthly revenue at risk of about 136,244 currency units (the sum of $P \times$ MonthlyCharges). Because the scoring pass includes customers used for training, these totals are somewhat optimistic; the `split` column in `predictions.parquet` allows the test-only subset to be reported separately.

Customer segmentation

K-Means with k = 4 gives a silhouette of 0.5252. Fewer clusters score slightly higher (k = 2: 0.5991, k = 3: 0.5900), so k = 4 was kept for its more useful business granularity, supported by a drop in WSSSE from 13,380 to 10,263. The two PCA components explain 88.1% of the variance of the clustering features. The segments are very different in churn behaviour:

- **Cluster 0 – New low-spend month-to-month customers:** 2,115 customers, average tenure 8.2 months, monthly charge 46.08, 0.53 add-ons, churn **39.67%**.
- **Cluster 1 – Mid-tenure high-spend flexible-contract customers:** 2,116 customers, average tenure 25.6 months, monthly charge 81.68, 2.58 add-ons, churn **38.56%**.
- **Cluster 2 – Loyal low-spend customers on long contracts:** 1,066 customers, average tenure 45.8 months, monthly charge 25.55, 0.34 add-ons, churn **1.41%**.
- **Cluster 3 – Loyal premium multi-service customers:** 1,746 customers, average tenure 61.7 months, monthly charge 90.83, 4.25 add-ons, churn **11.4%**.

Clusters 0 and 1 together hold 4,231 customers with churn near 39% and are the natural targets for retention campaigns, whereas cluster 2 churns at only 1.41%.

MonthlyCharges regression

The OLS model reaches test $R^2 = 0.9989$, RMSE = 1.0275 and MAE = 0.7891; the elastic-net model (regParam 0.3, elasticNetParam 0.8) is slightly weaker ($R^2 = 0.9983$, RMSE = 1.2500, MAE = 1.0193) because regularisation shrinks the coefficients. The OLS coefficients show an almost perfectly additive tariff: fiber optic +50.05, DSL +25.07, phone service +20.06, streaming TV/movies about +10 each and every other add-on about +5, all with p-values below 0.001. The very high R^2 therefore reflects a deterministic price list rather than a hard prediction problem.

Spark SQL internals, Scala module and serving performance

The physical plan shows Spark reading only four columns from Parquet (column pruning) and aggregating in two phases around a hash-partitioned exchange. Caching reduced five repeated

aggregations from 1.74 s to 1.39 s (about 20%); the gain is modest because the dataset is tiny, but it grows with data size and with iterative algorithms. The Scala module confirms the SQL results with a typed Dataset, segments customers by a pattern-matched rule (Critical segment: 2,128 customers with 54.61% churn; Stable segment: 3.59%), and shows that a composable rule score of at least 0.8 flags 876 customers of whom 70.55% churned. The lazy `Stream` materialised only 47 events to find five dropped calls, illustrating the same deferred-execution idea as Spark's lazy DAG. The Express API answered every request in roughly 1–7 ms from memory, far inside the 50 ms target, without a database or a live Spark job.

Limitations

With only 7,043 rows a single-node Spark run is slower than pandas – the cross-validated Logistic Regression alone took 91 s – so the benefits of Spark are architectural here, not speed. The stratified split uses `sampleBy`, which is approximate (5,698 training / 1,345 test rows, 80.9% / 19.1%). Class imbalance limits recall, and the dataset is a single-snapshot sample, so results should not be generalised to other operators without re-validation.

12. Conclusion

This project designed, implemented and executed a complete Apache Spark analytics pipeline for telecom customer churn. PySpark and Spark SQL ingested and validated 7,043 customer records with an explicit schema, engineered six features without leakage, and produced a Parquet/JSON analytical mart. A leak-free Spark ML Pipeline with 3-fold cross-validation trained five classifiers; on an honest, unresampled test set they reached 77.6%–80.9% accuracy with ROC-AUC up to 0.837, and the Random Forest scored every customer into 856 High, 1,651 Medium and 4,536 Low risk groups.

Spark SQL showed that contract type, tenure, fiber-optic service and electronic-check payment are the dominant churn drivers; K-Means separated customers into four segments with churn rates from 1.4% to 39.7%; and Linear Regression recovered the additive pricing structure with $R^2 \approx 0.999$. The Scala companion demonstrated typed Datasets, immutable collections, pattern matching and lazy evaluation, and the Express API served all results from memory with millisecond latency and no database.

Overall, the project demonstrates that distributed in-memory processing, declarative query optimisation and pipeline-based machine learning can replace a database-centred prototype with a reproducible, portable and scalable architecture. Future work includes running on a multi-node cluster or cloud platform with larger CDR-style data, threshold tuning and class weighting to improve churn recall, Structured Streaming for real-time scoring, and model tracking with MLflow.